Nelson Open - Congruence Closure DAG - SAT Solver

Silver Gjeka

February 2, 2025

University: University of Verona (UNIVR)
Subject: Automated Reasoning
Degree: Master's Degree in Artificial Intelligence
Matriculated: VR527645

Contents

1 Introduction:

This project focuses on developing a solver that determines whether a set of logical formulas can be satisfied. The formulas belong to three logical theories: equality (with free symbols), non-empty possibly cyclic lists, and arrays (without extensionality). The core of the solver is a congruence closure algorithm applied to Directed Acyclic Graphs (DAGs). This algorithm checks if the given equalities and inequalities can be consistently satisfied.

To improve the performance and efficiency of the solver, we have incorporated several heuristics:

- **Forbidden List or Set**: This heuristic helps avoid problematic combinations of elements that may lead to contradictions or logical errors.
- **Choosing the Representative in the UNION Function**: When merging groups of elements, the solver selects the element with the largest set of related elements. This helps ensure better decision-making when performing merges.
- **Non-Recursive FIND Function**: This version of the FIND function eliminates recursion, which increases efficiency by ensuring that all terms in a group are updated correctly when merging.

These improvements may not always help in every situation, but they are meant to make the solver faster and more accurate. Since their effects can vary, you can choose to test them and see how they impact the solver's performance.

1.1 Formula Transformations

To handle a variety of formulas, the solver applies several transformations to simplify them before processing. If any unsupported symbols are found, an error will be raised, indicating that those symbols are not accepted.

- Free predicate symbols are adjusted where necessary to fit the solver's needs.
- Non-conjunctive formulas are converted to Disjunctive Normal Form (DNF), making them easier to process.
- If the formula contains symbols that are not supported, an error will be triggered, notifying that these symbols are not accepted.
- Quantifiers are removed, and variables are treated as free symbols, allowing the solver to handle them more easily.

These transformations ensure that the formulas are in a standardized format, making it easier for the solver to determine their satisfiability.

2 Project Functionality

The project is based on the Nelson-Oppen method, which handles all three theories in the formula, determining its satisfiability.

Warning

The input formulas must be written in *Negation Normal Form (NNF)* before being processed by the project. Incorrectly formatted formulas may result in errors or invalid results.

In this project, you will choose the following operations for your formula:

- *Disjunctive Normal Form (DNF) Transformation:* This option allows the input formula to be transformed into Disjunctive Normal Form (DNF), which simplifies the structure of the formula for easier analysis.
- *DAG-Based Congruence Closure Algorithm:* Using the Nelson-Oppen method, the project integrates three key logical theories: **Equality**, **Arrays**, and **Lists** to reason about the input formula. The algorithm constructs a Directed Acyclic Graph (DAG), applies congruence closure, and checks for satisfiability.

2.1 Syntax

Formulas consist of variables, atoms, predicates, list operations, functions, and array operations, connected with equality (`=`) or disequality (`!=`) symbols to evaluate satisfiability.

Here are some examples: *Variables:* x, y, z, *Atoms:* `atom(a)`, *Predicates:* $PRED($`atom`$(a))$, *List Operations:* `car(l)`, `cdr(l)`, `cons(h, t)`, *Array Operations:* `select(a, i)`, `store(a, i, v)`

- *Disjunctive Normal Form (DNF)* formulas include logical operations: OR (`|`), AND (`&`), NOT ($\sim$), Implication (`->`).
- *Congruent Closure DAG* use negation as '-' and conjunction as ';'.

Restrictions:

- Formulas must be well-formed, using only valid characters.
- Space operators: `a = b` not `a=b`.
- DNF formulas must have correct parentheses to define logical precedence.
- The bi-conditional (`<->`) must be written as $((a \rightarrow b) \wedge (b \rightarrow a))$.

2.2 Parsing & DNF:

- **Parser:**
  - *Step 1:* If any quantifiers are present, they will be removed. This means that universal quantifiers ($\forall$) and existential quantifiers ($\exists$) will be eliminated from the formula, simplifying the structure.

- *Step 2 & 3:* First, identify all the functions within the formula and map them to simpler terms to eliminate the complexity of the functions. Once the functions have been simplified, apply an additional mapping to the equalities and disequalities. This step ensures that the formula is reduced to a simpler form containing only logical operations such as OR (|), AND (&), and Implication ($\rightarrow$).

- **DNF:**
  - *Step 4:* Applying the truth table to the formula, we can derive the final result in DNF format.

  For more information on the DNF transformation, check the link: Disjunctive Normal Form on Wikipedia.

- **Parser:**
  - *Step 5:* Remapping the results on the result dnf formula
  - *Step 6:* Splitting the formula by OR(|)
  - *Step 7:* Applying De Morgan's Law when encountering negations of equalities.
  - *Step 8:* Saving the spliced formulas in different files.

- **Example:**
  - *Step 1:* Formula:
    $$\forall x \, \text{car}(x) = \text{select}(\text{store}(a, i, v)) \,|\, b \neq a$$
    Remove quantifiers:
    $$\text{car}(x) = \text{select}(\text{store}(a, i, v), i) \,|\, b \neq a$$
  - *Step 2 & 3:* Map functions:
    $$f_0 \rightarrow \text{car}(x), \quad f_1 \rightarrow \text{select}(\text{store}(a, i, v), i)$$
    Resulting in:
    $$f_0 = f_1 \,|\, b \neq a$$
    Map equalities and disequalities:
    $$e_0 \rightarrow f_0 = f_1, \quad e_1 \rightarrow b \neq a$$
    Result:
    $$e_0 \,|\, e_1$$
  - *Step 4:* Apply truth table to derive DNF:
    $$\text{DNF} = (e_0 \wedge e_1) \vee (e_0 \wedge \neg e_1) \vee (\neg e_0 \wedge e_1)$$
  - *Step 5:* Remap DNF terms with f_0 and f_1:
    $$(f_0 = f_1 \wedge b \neq a) \vee (f_0 = f_1 \wedge \neg(b \neq a)) \vee (\neg(f_0 = f_1) \wedge b \neq a)$$
  - *Step 6:* Split by OR (|) into:
    $$(f_0 = f_1 \wedge b \neq a), \, (f_0 = f_1 \wedge \neg(b \neq a)), \, (\neg(f_0 = f_1) \wedge b \neq a)$$

– *Step 7 & 8:* Save each part in separate files with De Morgan's Law applied:

* **File 1:**

$$\text{car}(x) = \text{select}(\text{store}(a, i, v), i); b \neq a$$

* **File 2:**

$$\text{car}(x) = \text{select}(\text{store}(a, i, v), i); \neg(b \neq a) \rightarrow \text{car}(x) = \text{select}(\text{store}(a, i, v), i); b = a$$

* **File 3:**

$$\neg(\text{car}(x) = \text{select}(\text{store}(a, i, v), i)); b \neq a \rightarrow \text{car}(x) \neq \text{select}(\text{store}(a, i, v), i); b \neq a$$

2.3 NELSON-OPPEN & Congruence Closure Algorithm DAG

The Nelson-Oppen and Congruence Closure Algorithm works with formulas stored in the `alreadyToDnf/` folder. This folder contains formulas that have been converted to Disjunctive Normal Form (DNF) as well as test files added manually. The algorithm reads one of these files to check if the formulas can be satisfied.

2.3.1 Parser

The parser processes the input formula by tokenizing it and constructing a Directed Acyclic Graph (DAG). During parsing, the formula is broken into tokens, and nodes are created to represent the formula's structure.

- The parser performs the following steps:

1. *Tokenization and Formula Parsing*: The input formula is tokenized into smaller components like variables, predicates, functions, and operators, allowing for easier processing.
2. *DAG Construction*: A Directed Acyclic Graph (DAG) is constructed, where each node represents an element of the formula. Nodes are linked together, and each node contains a *parent field*, a *find field* for equalities, and optional *arguments* for operations like `car`, `cdr`, `select`, etc.
3. *Predicate Sets*: The parser uses the following sets:
   - `equalPred`, `notEqualPred`, `atomPred`, and `consTerm` to store various predicates and terms.
4. *Error Handling*: If the formula contains invalid symbols or unsupported operations, the parser throws an exception, ensuring that the input is valid before proceeding with further steps.

2.3.2 Congruence Clousure Algorithm

- The algorithm is executed in several structured steps:

1. *Initialize the Algorithm and DAG*: The input formula is parsed, and a Directed Acyclic Graph (DAG) is constructed to represent its structure. Nodes are created from the smallest to the largest element in the formula during the parsing process.
2. *Atom Conflict Detection*: The algorithm examines all constructed atomic elements for conflicts.

3. *List Theory Conflict Check (Car, Cdr)*: The algorithm checks the car, cdr operations on the created list cons. If conflicts are found, they are reported as errors.

4. *Equality and Inequality Processing*: The algorithm evaluates equalities (`=`) and inequalities (`!=`).

   - If two elements are equal, their nodes are merged.
   - If a conflict arises (e.g., `a = b` and `a != b`), the formula is marked as unsatisfiable.

5. *Process Store Operations (Array Theory)*: The `store` operation is processed to ensure consistency in the array theory.

   - For example, `store(a, i1, v5)` guarantees that `select(a, i1)` returns `v5`.
   - If this condition is violated, a conflict is reported.

6. *Process Select Operations*: The `select` operation is processed to ensure that the values returned are consistent with the array operations. If there is an inconsistency in select results, it is flagged as a conflict.

7. *Final Satisfiability Check*: After processing all operations and transformations, the DAG is analyzed to determine whether the input formula is satisfiable. If no conflicts are detected, the formula is satisfiable. Otherwise, it is unsatisfiable.

2.4 Testing

Testing is evaluated on different complexity equality/list/array formulas (Fig. 1, 2), you can generate them by using the formula generator. (Check `README` how to use it).

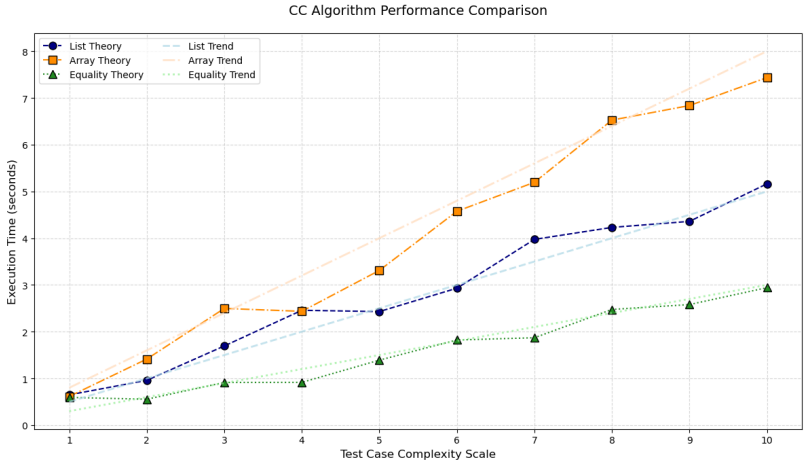

Figure 1: Time complexity analysis showing polynomial scaling (CC Algorithm)

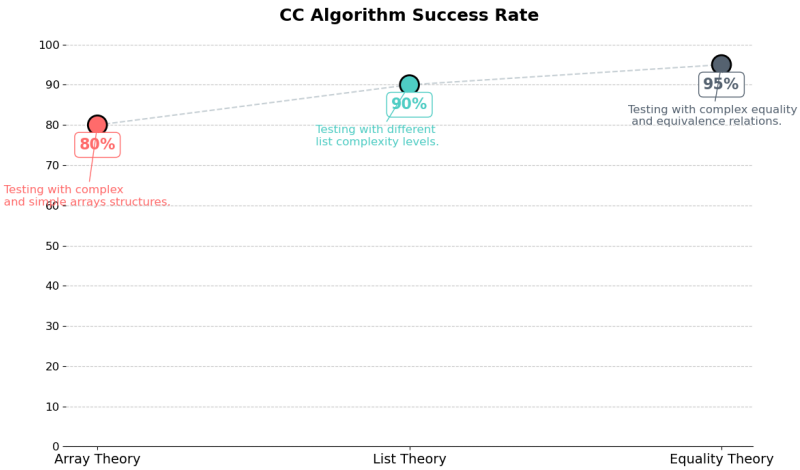

Figure 2: Correct prediction rates: Array (80%), List (90%), Equality (95%)

3 Conclusion

This project introduces a solver that checks whether logical formulas are satisfiable in three key areas: equality with free symbols, non-empty possibly cyclic lists, and arrays without extensionality. The solver works by transforming formulas and using a combination of the Nelson-Oppen method and a DAG-based congruence closure algorithm to efficiently solve the problem. Key improvements, like the forbidden list heuristic and a non-recursive FIND function, have been added to improve performance.

The solver processes formulas step-by-step, from parsing them to checking if they are satisfiable, ensuring reliable results while being computationally efficient. The project shows how combining theory with practical optimizations can create a strong solver that can handle many types of logical formulas. It also sets the stage for future updates, such as adding new logical theories or more advanced methods.

References

[1] Bradley, A.R., Manna, Z.: *The Calculus of Computation*. Springer (2007)

[2] Kroening, D., Strichman, O.: *Decision Procedures*. Springer (2008)

[3] Armando, A. et al.: Rewriting satisfiability. *Inf. Comput.* 183(2), 140–164 (2003)

[4] Bachmair, L. et al.: Abstract congruence closure. *J. Autom. Reason.* 31(2), 129–168 (2003)